

Exercices sur la calculabilité

1.1 Connaissance du cours

1. Parmi les langage suivants, y-en-a-t-il qui ne sont pas équivalents aux autres pour résoudre un problème ? C, python, java, html, la machine de Turing.
2. Que faut-il définir pour énoncer un problème ?
3. Quelles sont les différences de définition entre : *Problème*, *Fonction*, *Algorithme*.
4. Vrai/Faux : Il existe plus de problèmes que d'algorithmes permettant de les résoudre.
5. Qui est le premier à avoir énoncé la théorie de l'indécidabilité : Hilbert ou Gödel ?
6. Quel est le théorème énoncé par Turing sur le problème de la terminaison d'un algorithme ?

1.2 une fonction termine

Supposons qu'il existe une fonction calculable termine(fonction, données) qui prend 2 arguments :

- une fonction,
- et des données d'entrée pour cette fonction

Cette fonction termine renverrait True si le programme termine et False s'il entre dans une boucle infinie.

On définit les deux fonctions suivantes :

```
1 def fonction1(n):
2     if n % 3 != 0:
3         return True
4     else:
5         return False
```

```
1 def fonction2(n):
2     while n % 3 != 0:
3         print("True")
4     print("False")
```

1. Cette fonction termine, peut-elle réellement exister ?
2. Que renverront les appels suivants ?

```
1 termine(fonction1, 7)
2 termine(fonction1, 9)
3 termine(fonction2, 7)
4 termine(fonction2, 9)
```

Justifier vos réponses.

1.3 Recherche du maximum

La fonction maximum renvoie le maximum d'une liste :

```
1 def maximum(L):  
2     M = L[0]  
3     for k in range(1, len(L)):  
4         # à compléter  
5     return M
```

1.3.1 Compléter le code (ligne 4) en choisissant parmi les propositions :

- a. `if M < L[k] : M = L[k]`
- b. `if M > L[k] : M = L[k]`
- c. `if M == L[k] : M = L[k]`

On cherche maintenant à PROUVER cette fonction.

1.3.2 cette fonction, termine t-elle toujours ? Démontrez le.

1.3.3 Pour montrer que cette fonction fait bien ce que l'on attend d'elle, on exhibe un invariant de boucle : "M est le maximum de la liste L[:k]". Démontrer cet invariant.

1.4 Multiplication égyptienne

Considérons le programme suivant, qui implémente un ancien algorithme égyptien. Dans cet exercice, a et b sont supposés être des entiers positifs.

```
1 def Egyptienne(a, b):  
2     t = 0  
3     while a > 0:  
4         if a % 2 == 1:  
5             t = t + b  
6             b = 2 * b  
7             a = a // 2  
8     return t
```

1. Montrer que la fonction `Egyptienne` termine toujours.
2. Détailler l'exécution de `Egyptienne(41, 3)`.
3. Montrer que la quantité $a*b+t$ est un invariant de boucle.

Difficilement calculable

Nous avons vu qu'il existe des problèmes **non calculables** (comme le problème de l'arrêt). Mais parmi les problèmes calculables, certains sont en pratique **impossibles à résoudre** : non pas parce qu'il n'existe pas d'algorithme, mais parce que cet algorithme prendrait... des millions d'années.

Dans cet exercice, on explore cette frontière.

2.1 Partie 1 — Compter avant de coder

Pour chacun des problèmes suivants, on cherche d'abord à **dénombrer le nombre de cas à examiner** si on teste toutes les possibilités (méthode exhaustive).

2.1.1 Problème A — Le verrou à combinaison

Un cadenas a n chiffres (de 0 à 9 chacun). On cherche le bon code par essais successifs.

Q1. Combien de combinaisons faut-il tester au maximum pour $n = 3$? Pour $n = 6$? Pour $n = 10$?

Q2. Exprimer le nombre de combinaisons en fonction de n . Quelle est la complexité de la recherche exhaustive?

2.1.2 Problème B — Le sac à dos

On dispose de n objets. Pour chaque objet, on choisit de le prendre (1) ou non (0).

Q1. Lister toutes les combinaisons possibles pour $n = 3$ objets (on pourra utiliser des suites de 0 et 1).

Q2. Combien y a-t-il de combinaisons pour $n = 10$? Pour $n = 30$? Pour $n = 100$?

Compléter le tableau :

n	Nombre de combinaisons
10	
20	
30	
50	
100	

Q3. Sachant qu'un ordinateur peut tester environ 10^9 combinaisons par seconde, estimer le temps de calcul pour $n = 50$ et $n = 100$.

Quelle conclusion tirer?

2.1.3 Problème C – Le voyageur de commerce (version simplifiée)

Un livreur doit visiter n villes exactement une fois et revenir à son point de départ, en minimisant la distance totale parcourue.

Q1. Pour $n = 3$ villes, combien d'itinéraires différents (ordres de visite) existe-t-il ?
Les lister.

Q2. Pour $n = 4$ villes ? Pour $n = 5$?
Compléter :

n	Nombre d'itinéraires
3	
4	
5	
10	
20	

Q3. Exprimer le nombre d'itinéraires en fonction de n . Quelle notation de complexité correspond ?

2.2 Partie 2 – Mise en perspective

Q1. Classer les trois problèmes ci-dessus par ordre croissant de complexité (de la moins explosive à la plus explosive).

Q2. Parmi les complexités suivantes, associer chacune au bon problème :

$$O(n!) \quad O(2^n) \quad O(10^n)$$

Q3. Un problème de complexité $O(n^2)$ sur $n = 1\,000\,000$ données nécessite environ 10^{12} opérations – ce qui est déjà très long. Que dire alors d'un problème en $O(2^n)$ pour $n = 100$?
(On rappelle que $2^{100} \approx 10^{30}$, et que l'univers a environ 4×10^{17} secondes.)

2.3 Partie 3 – Et pourtant, on peut faire mieux...

Pour le **problème du sac à dos**, on admet qu'il existe un algorithme qui calcule la solution optimale avec une complexité de $O(n \cdot W)$, où W est la capacité maximale du sac.

Q1. Si $n = 100$ objets et $W = 500$ kg, combien d'opérations cet algorithme nécessite-t-il ?
Comparer avec la méthode exhaustive.

Q2. Cet algorithme miraculeusement plus efficace repose sur une idée clé : **ne pas recalculer ce qu'on a déjà calculé**, en stockant les résultats intermédiaires dans un tableau.
Cette technique s'appelle _____.

Q3. Cette technique permet-elle de résoudre *tous* les problèmes de complexité exponentielle ?
Pourquoi n'a-t-on pas pu faire pareil pour le problème de l'arrêt ?

Le prochain cours présente cette technique et les conditions dans lesquelles elle s'applique.